大世界道路贴地渲染性能研究

Executive Summary

你现在这套实现之所以贵，不是“道路贴图天生就贵”，而是因为它在 `chunk mount` 时用 CPU 逐块重建道路 mask，再把结果上传成纹理。这和业界成熟方案的差异很大。成熟做法通常是把“道路数据”和“道路表面着色”分层处理，并尽量采用预烘焙、缓存、虚拟纹理、地形权重图或近远分层，而不是每次进 chunk 都在主线程重画一遍 [1][2][3][5]。

结合你当前项目的 profiling，本地热点依然集中在 `ground_mask_textures_usec`。因此最适合你的方向不是“砍道路”，而是把道路地表表达改成“可缓存、可分层、可增量更新”的体系。短期最优解是懒生成并持久化 chunk road mask；中期最优解是把路面下沉到 terrain control map / basemap LOD；长期最成熟的方向则是 RVT 或 clipmap 式的分页地表覆盖 [1][3][4]。

Key Findings

- **成熟引擎不会在每次 chunk 挂载时主线程重绘整块道路 mask**：Unreal 把大面积地表着色和 spline/decal 类效果往 Runtime Virtual Texturing 上收敛，核心就是缓存和复用，而不是反复现场烘焙 [1]。
- **Terrain 权重图/控制图是道路“贴地但不是实体网格”的标准落点之一**：Unity 的 Terrain Paint API 直接面向纹理上下文和相邻地块无缝处理，这类体系天然适合道路写入 terrain layer，而不是把道路当独立 3D 物体 [2]。
- **近远分层是成熟方案的共同点**：Unity 的 terrain basemap 机制本质上就是远处用合成结果、近处用高细节层，说明“远处只保轮廓，近处补细节”是标准套路 [3]。
- **70km 级别世界通常需要分页纹理思路，而不是单 chunk 即时栅格化**：Geometry clipmaps/texture clipmaps 代表的大世界成熟路线，是用嵌套层级和局部更新覆盖超大地表 [4]。
- **Godot 里也不应该把 GPU 相关资源创建当成随意可并行的事情**：Godot 官方明确提醒与渲染服务器/独占资源/GPU 交互相关的内容要谨慎，线程里更适合做原始数据准备，主线程只做最终资源提交 [5]。

Detailed Analysis

1. 你当前实现为什么会贵

当前实现的关键路径是：

1. 读取 chunk 的 `road_segments`
2. 在 CPU 上构建 `road_mask_texture` / `stripe_mask_texture`
3. 创建 `Image`
4. 创建 `ImageTexture`
5. 把纹理绑定到地面 shader

本地 profiling 已经说明，真正的大头不是纹理上传，而是“在 CPU 上把道路涂进 mask”这一步。换句话说，你现在卡的不是“shader 太复杂”，而是“每次 mount 都在重新烘焙贴图”。

这在静态世界里尤其亏，因为你的道路网是确定性的。既然道路网不会在玩家每一帧行走时变化，就没有理由让每个 chunk 每次出现时都重新把整张道路贴图画一遍。

2. 业界成熟方法一：预烘焙或懒缓存 terrain control map

这是最贴近你当前结构、改动最小、收益最大的路线。

Unity Terrain Tools 的 `BeginPaintTexture` / `PaintContext` 路线，本质上就是把 terrain 的纹理层当成可写入、可复用的地表数据层，而且官方文档明确提到相邻 terrain tile 的无缝处理 [2]。这类体系说明，成熟做法一般是：

- 道路在数据层是折线 / spline / graph
- 道路在渲染层被烘焙进 terrain 的 control map / splatmap / weightmap
- 地形材质只负责采样，不负责临场重新生成整张图

对你现在的项目，这意味着最现实的短期路线是：

- 第一次遇到某个 chunk 时生成 `road_mask` 和 `stripe_mask`
- 直接落磁盘缓存
- 下次 mount 直接加载纹理，不再重算

这和你已经做成功的 `road_graph` 启动缓存是同一个思想，只不过这次缓存对象从 graph 变成了 surface mask tile。

3. 业界成熟方法二：近远分层，远处只保路面轮廓

Unity terrain 的 `Base Map Dist.` / basemap 思路很关键：远处并不继续维持全套高细节 terrain layer，而是使用合成后的较低成本表示 [3]。

这对你的道路系统非常重要，因为车道线、交叉口软边、细路肩这些信息并不需要在所有 chunk、所有距离都存在。成熟做法通常是：

- **远景**：只保留“哪里是路”的低频底色
- **中景**：保留路面宽度和主要轮廓，不一定保车道线
- **近景**：才补 stripe、路口细节、破损、井盖、箭头等

换成你的项目语境，我建议的分层是：

- `far` chunk：只采样低分辨率 `road_fill_mask`
- `mid` chunk：采样 `road_fill_mask`，但不生成 `stripe_mask`
- `near` chunk：才生成或加载 `stripe_mask`
- 桥梁保持 3D 实体，不受这一层策略影响

这是很标准的“视觉等价、成本不等价”的处理。

4. 业界成熟方法三：RVT / 虚拟纹理，把道路表面当分页地表信息

Unreal 的 Runtime Virtual Texturing 文档明确把它描述为缓存大面积 shading data 的机制，并指出它很适合 landscape shading、decal-like materials 以及 splines [1]。这和你的需求高度一致，因为你的普通道路本质上就是：

- 贴地
- 大范围
- 数据静态
- 只需要在表面表现，不需要全部变成实体几何

RVT 这套思路的核心不是某个 Unreal 专有 API，而是下面这件事：

- 世界表面数据按页或按 tile 分块
- 只维护玩家附近那一圈需要的高价值页
- 地形 shader 只采样缓存结果
- 道路 spline / 图层写入的是虚拟纹理页，而不是每次重建独立材质资产

你在 Godot 里不一定要完整复刻 RVT，但可以做一个“RVT-lite”版本：

- 将世界切成更大的 surface page，例如 512m 或 1024m
- page 内存里保存 `road_fill_mask` / `stripe_mask`
- 近圈 page 高分辨率，外圈 page 低分辨率
- chunk 只引用 page 的子区域，不单独生成自己的整张 mask

这会比“每个 256m chunk 都自带一套独立 mask”更接近成熟引擎思路。

5. 业界成熟方法四：clipmap 思路应对 70km 级世界

Geometry clipmaps / texture clipmaps 的核心价值，在于用固定规模、分层嵌套的更新窗口覆盖超大世界，而不是把世界切成海量彼此独立且同精度的静态块 [4]。

对你这个 `70km x 70km` 的目标，启发很直接：

- 不应该认为所有 chunk 的路面细节精度都相同
- 不应该让玩家附近和几十公里外采用同一套渲染表达
- 不应该把每次跨 chunk 都等价为“卸载一整套，重建一整套”

更成熟的思路是：

- 近圈维护高分辨率道路表面页
- 中圈维护低分辨率表面页
- 远圈只依赖 minimap / skyline / 雾效 / 地块色块

如果未来你要把城市继续做大，clipmap 式 surface overlay 比“海量 chunk 独立 mask”更容易继续扩。

6. Godot 里的现实边界

Godot 官方关于线程安全的文档提醒得很清楚：和渲染服务器、GPU 资源、独占资源有关的操作不能简单粗暴地全丢进多线程 [5]。

这对你意味着：

- 线程里可以做：道路 polyline 到 byte mask 的 CPU 栅格化、磁盘缓存读写、压缩/解压
- 主线程再做：`Image` / `ImageTexture` 的最终提交与材质绑定

也就是说，正确路线不是“把今天的主线程重建原样搬去线程里”，而是把系统改成：

- 数据准备异步
- 资源提交同步但轻量
- 同一份 mask 尽可能复用

Areas of Consensus

- **静态道路数据应尽量预烘焙、缓存或分页复用**，而不是每次出现都现场重建 [1][2][4]。
- **地面道路和桥梁应该分治**：桥梁是实体几何；普通贴地道路更适合 terrain layer / virtual texture / decal-like surface 表达 [1][2]。
- **近远分层是必须的**：远景不需要完整道路细节 [1][3][4]。
- **超大世界需要分页和层级**，而不是统一精度和统一更新成本 [1][4]。

Areas of Debate

- **RVT 还是 control map/splatmap**：如果世界完全静态，离线或懒缓存的 control map 往往更简单；如果未来想支持编辑、破坏、动态道路事件，RVT 风格会更强 [1][2]。
- **每个 chunk 一张 mask 还是多 chunk 共用 surface page**：chunk mask 简单直接；page 方案更接近成熟引擎、扩展性更好，但架构复杂度更高 [1][4]。
- **道路细节要放在 shader 还是放在贴图**：底色适合贴图；局部细节和近景标线也可以改成少量程序化 shader 细节，这要看你后续是否追求更高保真。

Recommended Path

Phase A：立刻做，风险最低

目标：先把 `chunk mount` 的主线程重建成本砍掉。

1. 给 `road_fill_mask` / `stripe_mask` 做磁盘缓存，key 绑定 `chunk_id + road_layout_signature + mask_version`
2. `far` / `mid` chunk 不加载或不生成 `stripe_mask`
3. `near` chunk 优先读缓存；缓存 miss 时后台生成 byte mask，主线程只提交纹理
4. 调整现有 shader，让没有 `stripe_mask` 时自动退化为纯路面底色

这是最适合你当前代码库的方案，我认为性价比最高。

Phase B：下一阶段升级

目标：把“每个 chunk 一套 mask”升级成“surface page”。

1. surface page 尺寸提升到 512m 或 1024m
2. chunk 改为引用 page 子区域 UV
3. 建立近中远三圈 page 分辨率

4. minimap 与 surface page 共用道路 tile 数据来源

这样可以减少重复边界和重复生成。

Phase C：长期成熟路线

目标：把道路地表表达做成接近 RVT / clipmap。

1. surface overlay 做分页缓存与层级 LOD
2. 道路图层和 terrain shading 解耦
3. 高架、匝道、桥梁继续保留独立 3D 几何
4. 普通道路更多作为“地表信息层”存在

这会更接近成熟大世界引擎。

Sources

[1] Epic Games, “Runtime Virtual Texturing in Unreal Engine,” official documentation. 说明 RVT 用于缓存大面积表面着色数据，适合 landscape、decal-like materials 与 splines。
https://dev.epicgames.com/documentation/en-us/unreal-engine/runtime-virtual-texturing-in-unreal-engine

[2] Unity Technologies, “TerrainPaintUtility.BeginPaintTexture,” official scripting API. 说明 terrain 纹理绘制通过 `PaintContext` 处理，并支持跨相邻 terrain tile 的无缝绘制。
https://docs.unity3d.com/ScriptReference/TerrainTools.TerrainPaintUtility.BeginPaintTexture.html

[3] Unity Technologies, “Terrain settings / Base Map Distance” and related TerrainData basemap documentation, official engine docs. 说明 terrain 远景使用 basemap 这类合成结果来降低运行成本。
https://docs.unity3d.com/2023.2/Documentation/Manual/terrain-OtherSettings.html and
https://docs.unity3d.com/6000.0/Documentation/ScriptReference/TerrainData-baseMapResolution.html

[4] Hugues Hoppe and Frank Losasso, “Geometry Clipmaps: Terrain Rendering Using Nested Regular Grids,” research/project page. 代表大世界 terrain/texture 层级分页更新的成熟路线。
https://hhoppe.com/proj/geomclipmap/

[5] Godot Engine Documentation, “Thread-safe APIs,” official documentation. 明确指出与渲染服务器、GPU 相关的操作要谨慎处理，线程更适合做数据准备而非直接做全部渲染资源提交。
https://docs.godotengine.org/en/stable/tutorials/performance/thread_safe_apis.html

Gaps and Further Research

- 还没有把 Unreal/Unity 的具体资源开销模型一一映射到 Godot 4.6 的 renderer 细节，这需要进一步原型验证。
- Godot 社区里对“RVT-like terrain overlay”的成熟现成方案并不统一，可能需要你自己做轻量版本。
- 如果你未来要支持道路编辑器、可破坏道路、动态施工区，则需要重新评估“缓存静态 mask”与“动态写入 page”的取舍。